

Experiment No: 1

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. **Hash Generation using SHA-256:**
 - o Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.
2. **Target Hash Generation with Nonce:**
 - o Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**
 - o Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:**
 - o Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output

****Program #1: Python program that uses the hashlib library to create the hash of a given string:****

```
import hashlib                                     #error
statement

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()          #error
statement
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)                       #error
statement
```

Enter a string: Nikita Khushalani

Hash: 7a91dc60da229ea7388e5d628845871bb8407f4bd2ff02e40aa707d9fed29a39

****Program #2: Program to generate required target hash with input string and nonce****

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce                #
Error
```

```

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)
Error

```

****Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string****

3 Error

```

import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
            return nonce

        nonce += 1

# Get user input
input_string = "Nikita Khushalani"
num_zeros = 4

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

```

```

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
#Error

```

Hash: 0000546b00b39831985e775507c370f8b7085c6105ccbd6f83eec672008d56ca

Input String: Nikita Khushalani

Leading Zeros: 4

Expected Nonce: 18594

****Program 4: Generating Merkle Tree for given set of Transactions****

```

import hashlib
def build_merkle_tree(transactions):                                     # Error
    if len(transactions) == 0:
        return None

    # Hash each transaction before combining
    transactions = [
        hashlib.sha256(transaction.encode('utf-8')).hexdigest()
        for transaction in transactions
    ]

    print("Transaction Hashes:")
    for i, transaction_hash in enumerate(transactions, 1):
        print(f"T{i}: {transaction_hash}")

    # Recursive construction of the Merkle Tree
    while len(transactions) > 1:
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])

        new_transactions = []
        for i in range(0, len(transactions), 2):
            combined = transactions[i] + transactions[i+1]
            hash_combined =
            hashlib.sha256(combined.encode('utf-8')).hexdigest()

```

```
new_transactions.append(hash_combined) #  
Error
```

```
transactions = new_transactions
```

```
print("\nIntermediate Hashes:")  
for i, transaction_hash in enumerate(transactions, 1):  
    print(f"Hash {i}: {transaction_hash}")
```

```
return transactions[0]
```

```
# Example usage
```

```
transactions = [  
    "Rohan -> Meera : $347",  
    "Meera -> Arjun : $829",  
    "Arjun -> Tanya : $156",  
    "Tanya -> Kabir : $673",  
    "Kabir -> Rohan : $421"  
]
```

```
merkle_root = build_merkle_tree(transactions)  
print("\nMerkle Root:", merkle_root) #
```

```
Error
```

```
Transaction Hashes:
```

```
T1: 306155007b99b724f13085faa13b1101270d41020ffef1d7b141de3899e35f56  
T2: b472c84963410f0a54936feed16f336710f9951e6657df8216f3b72ad2abcefa  
T3: 4dcc793bdc906c24a30a2699b71b7d923187d5b12468eda24adlee2f6bf2148c  
T4: 9ee40ac250bb80acce92b9834edb80e68035b46b9049411c2c0815eed763a94f  
T5: c65554d3bc926b4b28f3a37c0ee67484c8f3fe6e4c244902bbb0ea0636c7781c
```

```
Intermediate Hashes:
```

```
Hash 1: f73705ee5a6a370151f1e85d8ccb7e74d319aade42b5b7985039cfb32d13bab8  
Hash 2: 03d41e2640798e6fa265ebcb3a1bd4c6adecea87abbcf3f797d5a31a322f6b33  
Hash 3: b6e5bd570c5b1279b2f2c4fa5a941e583c07caf5c31dcc69bc124ca0cd3123e
```

```
Intermediate Hashes:
```

```
Hash 1: 8b227b4812915c3bdd85a29359f3991ff92f771eed967d903a86ce4be20a7638  
Hash 2: 1e90dd46787d7c68ec734d228b4c650bd72490a934f64124f838ac87aa9f1862
```

Intermediate Hashes:

Hash 1: 02cac9835a7989449f160ad5e1e6983b0e5c37e101b91db56746bab0989c32f

Merkle Root:

02cac9835a7989449f160ad5e1e6983b0e5c37e101b91db56746bab0989c32f

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.